

A kind-of-opinionated capture-and-plan system.

I (Brendan) built this for myself, but the bones are general enough that someone with a similar workflow could plug in. This page is an honest tour - what it is, how the parts fit, and where it makes specific choices that may or may not match how you work.

QUICK NAV

- › [The three pieces](#)
- › [What it plugs into](#)
- › [What it does](#)
- › [How it thinks \(the conventions\)](#)
- › [A capture, end to end](#)
- › [Is this for you?](#)

The three pieces (and where they run)

This is the part that often surprises people. It's not one app - it's three loosely-coupled processes that share a single SQLite database. You install all three to get the full experience, but each one is small and self-contained.

1

MCP SERVER + SQLITE + SKILLS

The core (BriefCase + Kit)

Python MCP server registered at the user level in Claude Code. The agent (`/kit` or `/kit-lite`) is just a slash command that talks to it. Database lives at `~/.briefcase/briefcase.db` .

2

LOCAL FASTAPI DAEMON
(LAUNCHD)

The sidecar

Always-on background process at `127.0.0.1:8989` . Receives captures from the browser extension and writes them to the same DB the MCP server uses. Also runs the `/draft` endpoint that shells

3

CHROME MV3 EXTENSION

BriefCase extension

Right-click on any page (or hotkey) opens a compose popup. Sends captures to the sidecar with optional flags (Jira, code research, schedule a meeting, decision, etc.). Always-on, plays well with

Available from any directory you launch Claude Code in.

out to `claude -p` for in-browser drafting. Runs whether Claude Code is open or not.

Google Chat and any web page.

So yes, three installs. Setup goes: clone the repo, register the MCP at user level, run `install.sh` for the sidecar, load the unpacked Chrome extension. Once they're all up, you basically forget about them.

What it plugs into

BriefCase doesn't try to replace your existing tools - it tries to be the connective tissue between them. Here's what it reads from and writes to:

VAULT

ThriveNotes (Obsidian)

Reads project notes, writes meeting notes and weekly rollups. Each initiative gets a `Projects/<slug>/` folder. Kit never overwrites your hand-written notes - it appends or files alongside.

CALENDAR

Google Calendar (MCP)

Source of truth for events. Kit reads your calendar to plan days and propose meeting times. The DB only stores tasks - events stay in gcal where they belong.

TICKETS

Atlassian / Jira

Read and write. Kit can fetch tickets, create new ones, update epic descriptions, and link tickets back to local initiatives via the `external_refs` table.

CODE

Thriveworks repo (or any code repo)

When you launch Kit (or kit-lite) from inside a code repo, the working tools (`grep`, `git`, `gh`) are scoped naturally. Items tagged `needs_code_context` get picked up here.

PDLC

PDLC (read-only)

Kit reads project `context.yaml` files from `~/Programming/pdlc/` to align my local initiatives with the broader product roadmap. Kit never writes to PDLC - those go through PDLC's own slash commands.

VOICE

My voice profile

The `/draft` skill (and the sidecar `/draft` endpoint) reads `~/ .claude/rules/brendan-voice-profile.md` to write replies that actually sound like me. If you adopt this, you'd write your own voice profile.

What it actually does

CAPTURE

PLAN

Triage queue

Right-click anything in the browser, optionally tag it with flags (Jira, code research, schedule a meeting, decision, etc.), and send it to the queue. Or hit a keyboard shortcut to skip the right-click.

Daily planning

Calendar-first. Kit pulls your gcal events as the day's skeleton, then layers in tasks from the inbox. You confirm; it saves the daily plan.

CAPTURE

Brain dumps

Quick "don't want to forget this" capture. Optionally linked to an initiative, tagged for a specific week, or scheduled for a specific day - depending on how committed it is.

DECIDE

Decision log

Capture decisions as they happen, file them under an initiative. Later, in the relevant code repo, your dev agent reads the log and updates an in-repo `decisions.md`. Single source of truth, multiple consumers.

WRITE

Draft assistant (in your voice)

From the compose popup: "Draft now" button shells out to `claude -p` with your voice profile inlined. Returns a drafted reply in the popup. Copy-paste into Chat. No terminal copy artifacts.

WRITE

Stakeholder updates + retros

Pull initiative status across DB + Obsidian + Github (PRs / commits) and synthesize into stakeholder-ready text. Or generate a week-by-week retrospective with velocity trends.

SCHEDULE

Schedule a meeting

Capture a chat conversation, check "Schedule a meeting", paste attendee emails. Kit finds free slots across calendars, proposes 2-3 times, you pick one, it creates the event.

REVIEW

PR review queue

Tag captures with "PR review". When you walk the queue from the Thriveworks repo, those items are ready for `/review-as-brendan` or any review skill you've built.

PRINT

Thermal printer (optional)

Print today's checklist to a thermal receipt printer if you want a physical artifact for the day. Pure indulgence. Also prints custom markdown receipts.

COORDINATE

Multi-agent claim/release

Run Kit in BriefCase and kit-lite in your code repo simultaneously without stepping on each other. Atomic claim flips a queue item to `in_progress` with your label so the other session sees it's locked.

How it thinks (the conventions)

This is the part where you decide if the system fits your brain. I made specific choices about how work gets organized based on my personal workflows. They serve me well, but they may or may not serve you.

1. Events live in Google Calendar. Tasks live in BriefCase. Never both.

Calendar events have a single source of truth - your gcal. Kit never duplicates them into the DB. When planning a day, Kit pulls events from gcal, then layers in tasks from the inbox around them. If you're someone who likes a single unified todo list with calendar items inline, this won't feel right.

2. Three flavors of "I need to do this":

FLAVOR	WHEN	WHERE IT GOES
Brain dump	"Sometime, no specific day"	Inbox, no target_week. You'll triage it later.
Target week	"This week / next week, but not a specific day yet"	Inbox with target_week='2026-W19' . Surfaces during weekly planning.
Daily note	"Tuesday, do X"	Saved on the daily, not the inbox. It's already triaged.

The system asks which flavor you mean if it's ambiguous. The choice matters because each one shows up in a different surface (inbox / forecast / specific day).

3. Captures are signals, not decisions.

Sending something to the queue from the browser doesn't decide what to do with it. Triage decides. The flags you check at capture time (Jira, code research, schedule a meeting, decision, etc.) are *hints* for triage - they shape the conversation but never auto-execute.

4. Initiatives are the spine. Everything optionally hangs off one.

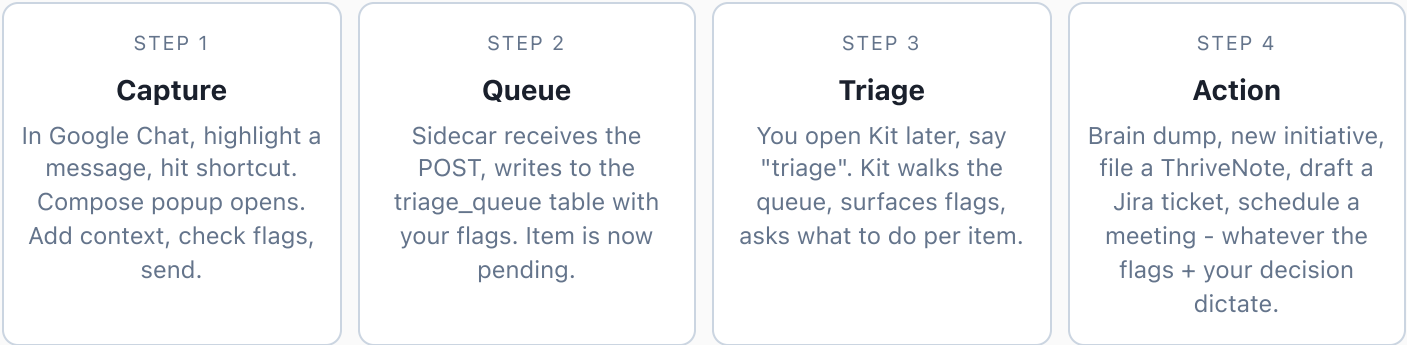
Inbox items, decisions, Jira tickets, ThriveNotes folders, and team members all attach to initiatives by slug. You can have orphan items, but the system encourages you to link them - that's how status reports and retros assemble themselves later.

5. Two Kits, two moods.

MODE	USE IT WHEN
<code>/kit</code> (full)	Daily planning, Monday briefing, weekly retro - anywhere you want the front-loaded context (calendar, forecast, recent activity, weekly rollup, PDLC alignment).
<code>/kit-lite</code>	Quick brain dump, walking the queue, code research from inside a repo. Same MCP, no front-loading. Pulls context only when you ask.

A capture, end to end

An example of how it actually flows from a chat message into structured work:



Concrete scenario. Dave messages you about a bug. You're in the middle of something else.

1. Highlight Dave's message, hit ⌘Y. Add: *"insurance-management - logs say we're matching plan name without state"*. Check **Code research** + **Jira ticket** (epic THRIV-13413). Send.
2. Item lands in the queue with both flags set.
3. Tonight, triage: Kit synthesizes the issue, drafts a Jira ticket body for your review, creates it under the IM Phase 3 epic, links it back as an `external_ref` on the resulting brain dump.
4. Brain dump is tagged `needs_code_context`. Tomorrow morning in the Thriveworks repo, kit-lite picks it up, you investigate the actual code together, file a fix.

Is this for you?

Probably yes if... • You already use Claude Code daily • You like keeping notes in a markdown vault (Obsidian or similar) • You want a single capture funnel for messages, web clips, decisions, and follow-ups	Probably not if... • You want a single all-in-one app with a real UI (this is mostly a CLI experience plus a browser extension) • You prefer todo-list tools where calendar events and tasks live in one combined feed
--	---

- You're comfortable with three small processes that don't talk to each other except through a shared SQLite file
- You have a few preferences about how planning should work and want them codified
- You're willing to write your own voice profile if you want the draft assistant

- You don't use Obsidian or any markdown notes system
- You'd rather not run a Python daemon, a Chrome extension, and a Claude Code MCP server all on one machine
- You want it to "just work" without learning a mental model of brain_dump vs target_week vs daily notes

If you're curious enough to try it

I can walk you through setup - it's three things to install (the MCP, the sidecar, the extension) and a couple of config files. The honest pitch: don't expect it to feel right immediately. The system has a few preferences baked in, and you'll either nod along or push back. Either's fine. The interesting question is whether the shape of how you work is roughly close enough to mine that adopting these conventions saves you more time than it costs you.

BriefCase + Kit. A personal tool I built to manage capture, planning, and decision tracking across the systems I use every day. Multiple components, single SQLite database, a few preferences baked in. Not a product - just useful enough that I figured I'd share it. Ping me with questions.